BIDMAS

Order of operations in JavaScript expressions

1 What BIDMAS Means



- Brackets
- Indices
- Division and Multiplication
- Addition and Subtraction

2 Evaluation Order

1. Start with brackets first
2. Then powers / exponents
3. Then multiplication and division from left to right
4. Finally addition and subtraction from left to right

```
2 + 3 * 4 = 14
(2 + 3) * 4 = 20
```

3 Why It Matters

- Prevents calculation mistakes
- Makes expressions predictable
- Helps you read code correctly

```
let total = 10 + 2 * 5; // 20
```

Key Idea

Use brackets when you want to change the default order.

INCREMENT AND DECREMENT

Changing a variable value by 1 in JavaScript

1 Increment (++)



- Adds 1 to a variable
- Common in counters and loops
- Can be written as prefix or postfix

```
let count = 5;
count++;   // 6
```

2 Decrement (--)



- Subtracts 1 from a variable
- Useful when values go down
- Also works as prefix or postfix

```
let lives = 3;
lives--;   // 2
```

3 Prefix vs Postfix

++x

x++

- Prefix changes first, then returns the value
- Postfix returns first, then changes the value

```
let x = 5; let a = ++x; // a=6
let y = 5; let b = y++; // b=5
```

Key Point

Use ++ and -- when you need simple step-by-step changes.

STRING OPERATORS

Working with text values in JavaScript

1 Concatenation with +



- Joins two strings together
- Creates one new text value
- Very common in beginner code

```
'Hello' + ' World'
→ Hello World
```

2 Concatenation Assignment (+=)



- Adds more text to an existing string
- Updates the original variable
- Useful for building messages step by step

```
let msg = 'Hi';
msg += ' Ahmad';
→ Hi Ahmad
```

3 Important Note



- String operators work on text values
- The + operator joins strings when either value is text
- Be careful when mixing strings and numbers

```
'5' + 2   // '52'
```

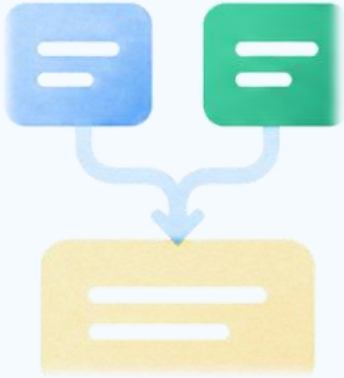
Remember

With strings, + usually means joining text, not mathematical addition.

MERGE STRINGS

Combining multiple text values into one string

1 Basic Merge



- Combine first text with second text
- Often used for names and messages
- Spaces must be added when needed

```
let fullName = firstName + ' ' + lastName;
```

2 Example Result

1. firstName = 'Ali'
2. lastName = 'Saleh'
3. Result becomes 'Ali Saleh'

firstName 'Ali'

lastName 'Saleh'

→ **Result** 'Ali Saleh'

3 Best Practice

- Keep strings readable
- Add separators like spaces or commas clearly
- Use template literals for more flexible formatting

```
'Ali' + ' ' + 'Saleh'
```

Key Idea

Merging strings means joining text pieces in the order you want them to appear.

TEMPLATE LITERALS

A modern way to build strings in JavaScript

1 What They Use



- Backticks instead of quotes
- ${ } to insert variables or expressions
- Easier to read than many + operators

```
`Hello ${name}`
```

2 Example

1. Store a name in a variable
2. Place it inside ${name}
3. JavaScript inserts the value automatically

```
let name = 'Sara';
let msg = `Hello ${name}`;
```

```
Hello Sara
```

3 Why Use Them

- Cleaner string building
- Great for multiline strings
- Good for mixing text with values

```
let message = `Total: ${price}
Thank you!`;
```

! Best Practice | **Template literals make dynamic strings clearer and easier to write.**

COMPARISON OPERATORS

Comparing values in JavaScript conditions

1 Common Operators



==	equal to
===	strict equal to
!=	not equal to
>	greater than
<	less than

2 Example Results

```
5 > 3          // true

5 < 3          // false

4 == '4'       // true

4 === '4'      // false
```

3 Why Strict Equality Matters

- == compares value after type conversion
- === compares both value and type
- Prefer === in modern JavaScript when possible

```
4 === 4    // true
4 === '4'  // false
```

Key Point

Comparison operators return true or false and are essential in conditions.